AI & LLM Engineering Guide

LangChain • LangGraph • RAG • Vector Databases • ChromaDB Embeddings • Prompt Engineering • LLM Basics • AI Agents • Multi-Agent Systems

A structured reference covering ten core topics in modern LLM application development — concepts, architecture, code examples, and practical guidance for building retrieval-augmented and agentic AI systems.

Table of Contents

101

LangChain

LangChain is an open-source framework for building applications powered by large language models. It provides standardized abstractions for chaining together prompts, models, tools, memory, and data sources so developers don't have to reinvent common LLM-application plumbing.

Core Components

- **Models** — unified interface to LLMs and chat models (OpenAI, Anthropic, Cohere, local models via Ollama, etc.)
- **Prompts** — PromptTemplate and ChatPromptTemplate objects for reusable, parameterized prompts
- **Chains** — sequences of calls (LLM + parser + tool) composed together; modern LangChain uses LCEL (LangChain Expression Language) with the | pipe operator
- **Memory** — stores conversation history (buffer memory, summary memory, entity memory) across turns
- **Retrievers** — pull relevant documents from a vector store for RAG pipelines
- **Tools & Agents** — let the LLM call external functions, APIs, or search engines
- **Output Parsers** — structure raw LLM text into JSON, Pydantic objects, lists, etc.
- **Document Loaders & Text Splitters** — ingest PDFs, web pages, CSVs and chunk them for indexing

Basic Example (LCEL)

```
from langchain_anthropic import ChatAnthropic
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

model = ChatAnthropic(model="claude-sonnet-4-6")
prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in two sentences."
)
parser = StrOutputParser()

chain = prompt | model | parser
result = chain.invoke({"topic": "vector databases"})
print(result)
```

When to Use LangChain

- Rapid prototyping of LLM pipelines with swappable components
- Standardizing across multiple model providers
- Building RAG pipelines with minimal boilerplate
- Integrating tool-calling agents with many pre-built connectors

Trade-offs: *LangChain's abstractions add convenience but also indirection — debugging deeply nested chains can be harder than writing raw API calls. Many teams use it for prototyping and drop to raw SDK calls for performance-critical production paths.*

102

LangGraph

LangGraph is a library (built by the LangChain team) for building stateful, multi-step LLM applications as graphs rather than linear chains. It models an application as nodes (functions/agents) and edges (control flow), with a shared state object passed between steps — making it well-suited for agents, loops, branching logic, and multi-agent orchestration.

Key Concepts

- **State** — a typed object (often a TypedDict or Pydantic model) shared and updated across the graph
- **Nodes** — Python functions or LLM calls that read and update state
- **Edges** — define transitions between nodes; can be conditional (branching based on state)
- **Cycles** — unlike simple DAG-based chains, LangGraph graphs can loop, enabling iterative reasoning, retries, and agent 'think-act-observe' loops
- **Checkpointing** — built-in persistence lets graphs pause, resume, and support human-in-the-loop review
- **StateGraph** — the primary class used to define nodes/edges and compile a runnable graph

Basic Example

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class AgentState(TypedDict):
    question: str
    answer: str

def research_node(state: AgentState) -> AgentState:
    state["answer"] = f"Researched answer for: {state['question']}"
    return state

graph = StateGraph(AgentState)
graph.add_node("research", research_node)
graph.set_entry_point("research")
graph.add_edge("research", END)

app = graph.compile()
result = app.invoke({"question": "What is RAG?", "answer": ""})
```

LangChain vs. LangGraph

Aspect	LangChain	LangGraph
Structure	Linear/DAG chains (LCEL)	Graph with cycles & branching
Best for	Simple pipelines, RAG, tool calls	Agents, loops, multi-step reasoning
State handling	Implicit / memory objects	Explicit shared state object
Human-in-loop	Limited	Native support via interrupts/checkpoints

103

RAG (Retrieval-Augmented Generation)

RAG is a technique that improves LLM outputs by retrieving relevant information from an external knowledge source at query time and inserting it into the prompt, rather than relying solely on the model's parametric (trained-in) knowledge. This reduces hallucination, allows use of up-to-date or private data, and doesn't require retraining the model.

Typical RAG Pipeline

- **1. Ingestion** — load documents (PDFs, web pages, DBs) and split them into chunks
- **2. Embedding** — convert each chunk into a vector using an embedding model
- **3. Indexing** — store vectors + metadata in a vector database (e.g. ChromaDB, Pinecone, Weaviate)
- **4. Retrieval** — at query time, embed the user's question and fetch the top-k most similar chunks
- **5. Augmentation** — insert retrieved chunks into the prompt as context
- **6. Generation** — the LLM generates an answer grounded in the retrieved context

Simplified Example

```
# 1. Retrieve relevant chunks
docs = vectorstore.similarity_search(query, k=4)
context = "\n\n".join(d.page_content for d in docs)

# 2. Augment the prompt
prompt = f"""Answer using only the context below.
Context:
{context}

Question: {query}"""

# 3. Generate
answer = llm.invoke(prompt)
```

RAG Variants

- **Naive RAG** — single retrieval pass, top-k chunks stuffed into the prompt
- **Re-ranking RAG** — retrieve a larger candidate set, then re-rank with a cross-encoder for precision
- **Hybrid Search** — combine dense (vector) search with sparse keyword search (BM25) for better recall
- **Query Rewriting / HyDE** — rewrite or expand the user query before retrieval to improve match quality
- **Agentic RAG** — the model decides iteratively whether to retrieve again, refine the query, or answer
- **Graph RAG** — retrieval over a knowledge graph instead of (or alongside) flat vector chunks

Why it matters: *RAG is the dominant pattern for grounding LLMs in private, domain-specific, or frequently changing data (support docs, internal wikis, legal filings) without the cost of fine-tuning.*

104

Vector Databases

A vector database stores high-dimensional embedding vectors and supports fast similarity search (nearest-neighbor lookup) over them. It's the backbone of semantic search and RAG systems, since it lets applications find content that is conceptually similar to a query, not just keyword-matching.

How Similarity Search Works

- Each document/chunk is converted to a vector (embedding) that captures its meaning
- A query is embedded the same way
- The database finds vectors closest to the query vector using a distance metric — cosine similarity, dot product, or Euclidean (L2) distance
- Approximate Nearest Neighbor (ANN) algorithms like HNSW (Hierarchical Navigable Small World) or IVF make this fast at scale (millions+ of vectors)

Popular Vector Databases

Database	Type	Notable Traits
ChromaDB	Open-source, embedded/local	Lightweight, great for prototyping
Pinecone	Managed cloud service	Fully managed, scales easily
Weaviate	Open-source / managed	Built-in hybrid search, GraphQL API
Milvus	Open-source, self-hosted	High scale, production-grade
Qdrant	Open-source / managed	Rust-based, fast, filtering support
FAISS	Library (not a DB)	Facebook's ANN search library, in-memory
pgvector	Postgres extension	Adds vector search to existing Postgres DBs

Key Selection Criteria

- Scale (thousands vs. hundreds of millions of vectors)
- Hosting model — self-hosted vs. managed/serverless
- Metadata filtering support (e.g. filter by date, user, category alongside similarity)
- Hybrid search (dense + sparse/keyword)
- Latency and indexing algorithm (HNSW is the current standard for most use cases)

105

ChromaDB

Chroma is an open-source, developer-friendly vector database designed to be embedded directly into applications with minimal setup. It's one of the most commonly used vector stores for prototyping RAG systems because it can run in-process (no separate server) or as a client-server deployment.

Key Features

- Runs in-memory, persisted to disk, or as a standalone server
- Built-in embedding functions (or bring your own — OpenAI, Anthropic-compatible, HuggingFace, Sentence-Transformers)
- Collections API for organizing documents into named groups
- Metadata filtering combined with similarity search
- Native integrations with LangChain and LlamaIndex

Basic Usage

```
import chromadb

client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_or_create_collection(name="docs")

# Add documents (Chroma embeds them automatically)
collection.add(
    documents=["LangChain simplifies LLM app development.",
               "ChromaDB stores and searches vector embeddings."],
    ids=["doc1", "doc2"],
    metadatas=[{"topic": "framework"}, {"topic": "database"}]
)

# Query
results = collection.query(
    query_texts=["What is a vector database?"],
    n_results=2
)
print(results["documents"])
```

When to Choose ChromaDB

- Local development, prototypes, and small-to-medium production apps
- You want zero external infrastructure to get started
- You need tight integration with LangChain/LlamaIndex pipelines
- For very large-scale, high-throughput production needs, teams often migrate to Pinecone, Milvus, or Qdrant

106

Embeddings

An embedding is a numerical vector representation of text (or images, audio, etc.) produced by a neural network, positioned in a high-dimensional space such that semantically similar inputs map to nearby vectors. Embeddings are the foundation of semantic search, RAG, clustering, recommendation, and classification tasks.

How Embeddings Are Generated

- A trained embedding model (e.g. OpenAI text-embedding-3, Voyage AI, Cohere Embed, or open-source Sentence-Transformers) processes text into a fixed-length vector (commonly 384–3072 dimensions)
- Training objectives (like contrastive learning) push similar meanings closer together and dissimilar meanings further apart in vector space
- The same model must be used for indexing and querying, since different models produce incompatible vector spaces

Distance / Similarity Metrics

Metric	Description
Cosine similarity	Measures the angle between vectors; ignores magnitude — most common for text
Dot product	Similar to cosine but sensitive to vector magnitude; used when vectors are normalized
Euclidean (L2) distance	Straight-line distance between points; common in image embeddings

Example: Generating an Embedding

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
vector = model.encode("LangChain simplifies LLM development.")
print(len(vector))  # e.g. 384 dimensions
```

Practical Notes

- Chunk size matters — embedding very long text averages out meaning and hurts retrieval precision
- Domain-specific fine-tuned embedding models can outperform general-purpose ones for specialized corpora
- Embedding cost and latency scale with the number and length of documents, so batching is important

107

Prompt Engineering

Prompt engineering is the practice of designing inputs to an LLM to reliably elicit the desired output — improving accuracy, format compliance, and reasoning quality without changing the underlying model.

Core Techniques

- **Clear, specific instructions** — state the task, format, and constraints explicitly
- **Few-shot prompting** — include example input/output pairs to demonstrate the desired pattern
- **Chain-of-thought (CoT)** — ask the model to reason step-by-step before giving a final answer, improving performance on multi-step problems
- **Role prompting** — set a system role/persona ('You are an expert data analyst...') to steer tone and focus
- **Structured output prompting** — request JSON, XML tags, or a specific schema so the response is machine-parseable
- **Self-consistency** — sample multiple reasoning paths and take the majority answer for higher-confidence results
- **ReAct (Reason + Act)** — interleave reasoning steps with tool calls, common in agent prompting
- **Negative examples** — show what NOT to do alongside positive examples to sharpen boundaries

Example: Structured Prompt

```
System: You are a support-ticket classifier.
Classify the ticket into one of: billing, technical, account, other.
Respond only with valid JSON: {"category": "...", "confidence": 0.0-1.0}

User: "I was charged twice for my subscription this month."
Assistant: {"category": "billing", "confidence": 0.97}
```

Common Pitfalls

- Vague instructions leading to inconsistent outputs
- Overloading a single prompt with too many tasks at once
- Not specifying output format, causing brittle downstream parsing
- Ignoring context window limits when stuffing in examples or retrieved documents

108

LLM Basics

Large Language Models (LLMs) are neural networks — almost always based on the Transformer architecture — trained on massive text corpora to predict the next token in a sequence. This simple objective, at sufficient scale, gives rise to broad language understanding, reasoning, and generation capabilities.

Key Concepts

- **Tokens** — text is broken into sub-word units (tokens); models process and generate one token at a time
- **Transformer architecture** — relies on self-attention to weigh the relevance of every token to every other token in a sequence
- **Context window** — the maximum number of tokens (input + output) a model can attend to at once
- **Pretraining** — the model learns general language patterns by predicting next tokens over huge datasets
- **Fine-tuning / RLHF** — further training (often with human feedback) aligns the model to follow instructions and be helpful/safe
- **Temperature & sampling** — parameters like temperature, top-p, and top-k control the randomness/creativity of generated text
- **Inference** — the process of running a trained model to generate output; distinct from training

Model Lifecycle

Stage	Purpose
Pretraining	Learn general language patterns from massive text corpora
Supervised fine-tuning (SFT)	Teach instruction-following on curated examples
RLHF / preference tuning	Align outputs with human preferences (helpful, harmless, honest)
Inference	Deploy the trained model to answer real user queries

Common Parameters

- **Temperature** — higher = more random/creative; lower = more deterministic/focused
- **Max tokens** — caps the length of the generated response
- **Top-p (nucleus sampling)** — restricts sampling to the smallest set of tokens whose cumulative probability exceeds p
- **System prompt** — sets persistent behavior/instructions that apply across the whole conversation

109

AI Agents

An AI agent is an LLM-driven system that can autonomously plan, decide, and take actions — such as calling tools, querying APIs, or writing code — in order to accomplish a goal, rather than simply producing a single text response.

Core Agent Loop

- **1. Observe** — receive the current state, task, or user input
- **2. Think/Plan** — reason about what action to take next (often via chain-of-thought)
- **3. Act** — call a tool, run code, or query an external system
- **4. Observe result** — read the tool's output and update understanding
- **5. Repeat** until the goal is achieved or a stopping condition is met

This pattern is often called the **ReAct** loop (Reason + Act) and is the foundation for most agent frameworks.

Key Components of an Agent System

- **Tools** — functions the agent can invoke (web search, code execution, database queries, APIs)
- **Memory** — short-term (conversation context) and long-term (persisted facts, vector-store recall)
- **Planner** — decomposes a complex goal into smaller steps
- **Executor** — carries out individual steps and handles tool calls
- **Reflection/self-critique** — some agents review their own output and retry if it's insufficient

Simplified Tool-Calling Example

```
tools = [{
    "name": "get_weather",
    "description": "Get current weather for a city",
    "input_schema": {"type": "object",
                     "properties": {"city": {"type": "string"}}}
}]

response = client.messages.create(
    model="claude-sonnet-4-6",
    tools=tools,
    messages=[{"role": "user", "content": "What's the weather in Tokyo?"}]
)
# The model returns a tool_use block; you execute get_weather("Tokyo")
# and send the result back for the model to produce a final answer.
```

Common Frameworks

- LangGraph, LangChain Agents, LlamaIndex Agents, AutoGen, CrewAI, OpenAI's Assistants/Agents SDK

110

Multi-Agent Systems

A multi-agent system coordinates several specialized AI agents — each with a distinct role, tool set, or area of expertise — to collaboratively solve a task that would be harder for a single generalist agent to handle well.

Why Use Multiple Agents

- **Specialization** — a researcher agent, coder agent, and reviewer agent can each be prompted/tuned for their specific job
- **Parallelism** — independent subtasks can run concurrently, reducing latency
- **Modularity** — easier to debug, replace, or improve one agent without redesigning the whole system
- **Better task decomposition** — a manager/orchestrator agent breaks a complex goal into subtasks assigned to worker agents

Common Architectural Patterns

Pattern	Description
Orchestrator–Worker	A central 'manager' agent plans and delegates subtasks to specialized worker agents, then aggregates r
Sequential pipeline	Agents run in a fixed order, each consuming the previous agent's output (e.g. researcher → writer → edit
Debate / critique	Two or more agents propose and critique each other's answers to converge on a better final result
Hierarchical teams	Sub-teams of agents report to team-lead agents, which report to a top-level coordinator
Peer-to-peer / group chat	Agents communicate in a shared thread, deciding amongst themselves who acts next (e.g. AutoGen grou

Example: Orchestrator Pattern with LangGraph

```
def orchestrator(state):
    state["subtasks"] = plan(state["goal"])
    return state

def researcher(state):
    state["research"] = search_tool(state["subtasks"]["research"])
    return state

def writer(state):
    state["draft"] = generate(state["research"])
    return state

graph = StateGraph(TeamState)
graph.add_node("orchestrator", orchestrator)
graph.add_node("researcher", researcher)
graph.add_node("writer", writer)
graph.add_edge("orchestrator", "researcher")
graph.add_edge("researcher", "writer")
graph.set_entry_point("orchestrator")
app = graph.compile()
```

Challenges

- Coordination overhead and increased latency/cost from multiple LLM calls
- Error propagation — a mistake in one agent's output can cascade to downstream agents
- Harder to debug and evaluate than a single-agent pipeline
- Requires careful design of communication protocols and shared state/memory

Summary: *Multi-agent systems trade added complexity and cost for better task decomposition, specialization, and (in some designs) parallel execution — most valuable for complex, multi-step workflows like research reports, software engineering tasks, or business process automation.*